

FAKE NEWS DETECTION SYSTEM

Machine Learning Based Web Application

Developed by: Deepanshu

B.Tech Project • Python • Flask • Scikit-Learn

1. Introduction

What is the project?

A web application that predicts whether user-provided news text is Real or Fake using machine learning.

How does it work?

The input is cleaned, converted into TF-IDF features, passed to a trained classifier, and the prediction is shown through a Flask interface.

Core ML Pipeline

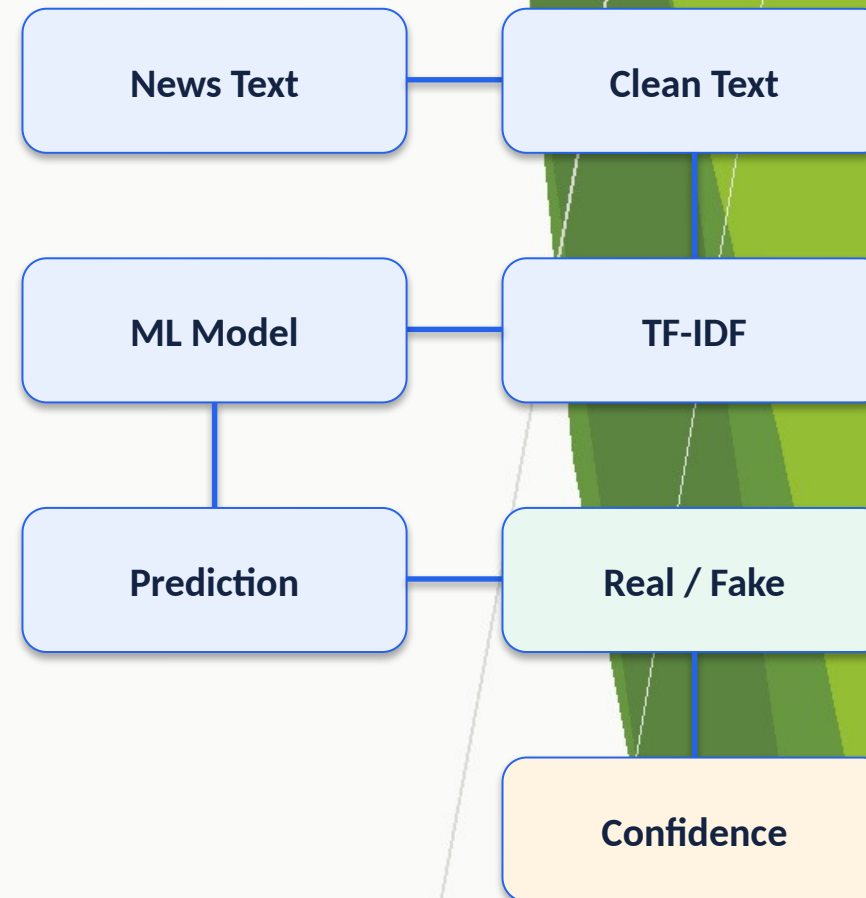
Text cleaning → TF-IDF vectorization → trained classifier → prediction.

Application Features

Real/Fake classification, confidence score, responsive interface, About page and Contact page.

2. Problem Statement

- False or misleading news can spread rapidly through digital platforms.
- Manual verification is time-consuming and difficult at large scale.
- Users need a simple tool that can analyze news text and provide an immediate prediction.
- The project addresses this by applying NLP-based feature extraction and machine learning.



3. Objectives of the Project

01 Detect

Classify submitted news text as Real or Fake.

02 Preprocess

Clean text by removing URLs, punctuation, numbers and extra spaces.

03 Transform

Represent text numerically using TF-IDF.

04 Predict

Use a trained linear classifier for fast prediction.

05 Explain

Display a confidence score with the prediction.

06 Deploy

Provide an easy-to-use Flask web interface.

4. Tools & Technology Requirements

Python

Core programming language

Pandas

Dataset loading and data handling

HTML / CSS / JavaScript

Frontend and responsive user interface

TF-IDF

Converts text into numerical features

Flask

Web framework for routes and prediction endpoint

Scikit-Learn

TF-IDF, model training, splitting and evaluation

Pickle

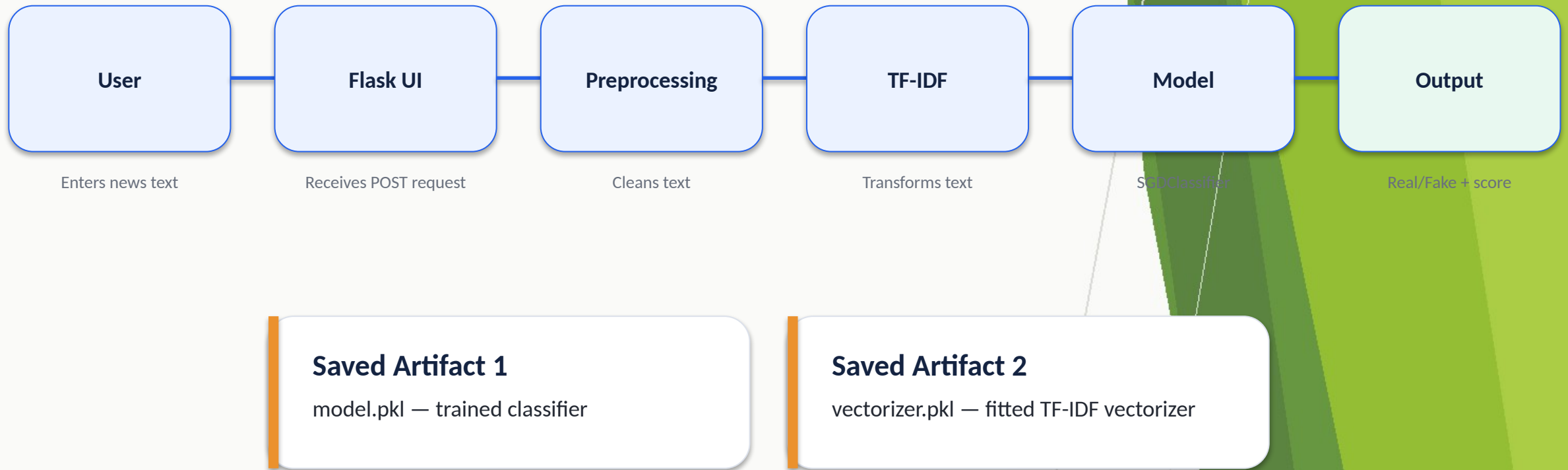
Stores and loads trained model/vectorizer

SGDClassifier

Classifier used in the supplied training code

5. System Architecture

End-to-End Flow



6. Machine Learning Workflow



- The supplied training code reads `dataset/news_dataset.csv` and expects text and label columns.
- An 80/20 train-test split is used with stratification and `random_state=42`.
- TF-IDF uses English stop-word removal and `max_df=0.7`.
- The supplied code trains `SGDClassifier(loss='hinge')` and reports accuracy on the test set.

7. Text Processing & Prediction

Step 1 — Input

User enters a news article/headline in the web form. Empty input is rejected.

Step 2 — Cleaning

Lowercase text; remove URLs, punctuation, numbers and extra spaces.

Step 3 — Vectorization

The saved TF-IDF vectorizer transforms cleaned text into numerical features.



8. Implementation / Main Modules

app.py

Flask routes: /, /about, /contact and /predict. Loads model.pkl and vectorizer.pkl and performs prediction.

train_model.py

Loads the dataset, cleans text, creates TF-IDF features, trains SGDClassifier, evaluates accuracy and saves artifacts.

model.pkl + vectorizer.pkl

Serialized trained classifier and fitted vectorizer used by the web application during prediction.

Frontend

templates/index.html, about.html and contact.html, supported by static CSS and JavaScript.

Dependencies

Pandas, Scikit-Learn and Flask are listed in requirements.txt.

9. Advantages

- Fast prediction after the model is trained.
- Simple and user-friendly web interface.
- Automated text preprocessing reduces manual effort.
- TF-IDF is efficient for traditional text classification.
- Model and vectorizer are saved and reused instead of retraining for every request.
- Provides a prediction confidence score when the model exposes `decision_function`.
- Easy to extend with additional models and features.

10. Limitations

- Prediction is based on text patterns learned from the training data; it does not independently verify facts.
- Performance depends strongly on dataset quality, balance and domain coverage.
- Current implementation is focused on text input rather than images, video or audio.
- The project is primarily English-oriented because TF-IDF is configured with English stop words.
- A linear classifier may not capture complex context, sarcasm or nuanced language as well as modern transformer models.
- The displayed confidence score is derived from the model decision score and should not be interpreted as a guaranteed probability of truth.

11. Future Scope

Deep Learning

LSTM/BERT-based models for stronger contextual understanding.

URL Detection

Accept a news URL and extract article content automatically.

Voice Input

Allow users to submit news through speech.

Explainable AI

Highlight words/features contributing to a prediction.

Multi-language

Support news in multiple Indian and international languages.

Live News API

Compare predictions with continuously updated news sources.

User Accounts

Login, prediction history and personalized dashboard.

Continuous Learning

Periodically retrain using verified, newly labeled data.

12. Conclusion

A practical ML-based approach to automated fake-news classification

The project combines NLP preprocessing, TF-IDF feature extraction, a trained linear classifier and a Flask web interface to provide quick Real/Fake predictions.



Thank You — Questions & Answers